



Universidad de las Fuerzas Armadas - ESPE

Departamento de Ciencias de la Computación

Tarea 1: Creación y ejecución de un proyecto

Carrera:

Ingeniería de Software

Asignatura:

Desarrollo de Aplicaciones Móviles

NRC: 30737

Integrantes:

Pilaguano Chisaguano David Alexander

Asmal Haro Kevin David

Villarroel Barreno Justin Joshua

Domínguez Aguilera Pablo Alejandro

Fecha: 30 de abril de 2026

Índice general

I.	Introducción	1
II.	Desarrollo	1
	II.1. Ejercicios	1
III.	Conclusiones	13
IV.	Referencias	14

Índice de figuras

1.	Estructura y main.dart.	1
2.	Ejecución y funcionamiento del aplicativo supermercado.	2
3.	Aplicativo funcionando sin ningun problema.	2
4.	validaciones de los campos.	3
5.	llenar los campos con datos validos y funcionamiento correcto.	3
6.	Estructura y main.dart.	4
7.	Ejecución y funcionamiento del aplicativo Máquina que puede dar vueltos.	4
8.	Aplicativo funcionando sin ningun problema.	5
9.	Validaciones de los campos.	5
10.	llenar los campos con datos validos y funcionamiento correcto.	6
11.	Estructura y main.dart.	7
12.	Ejecución del proyecto.	7
13.	Validación de entradas.	8
14.	Comprobación de año no bisiesto.	8
15.	Comprobación de año bisiesto.	9
16.	Código modelo ejercicio 4.	10
17.	Código controlador ejercicio 4.	10
18.	Código vista ejercicio 4.	10
19.	Ejecucion ejercicio 4	11
20.	Código modelo ejercicio 5.	12
21.	Código controlador ejercicio 5.	12
22.	Código vista ejercicio 5.	12
23.	Ejecucion ejercicio 5	13

Tarea 1: Creación y ejecución de un proyecto

I. Introducción

El desarrollo de aplicaciones móviles se ha convertido en una habilidad fundamental dentro del ámbito tecnológico actual, debido a la creciente demanda de soluciones digitales accesibles, eficientes y centradas en el usuario. En este contexto, la presente actividad tiene como objetivo la creación y ejecución de un proyecto de aplicación móvil, integrando conceptos clave como el diseño de interfaces (UI/UX), la lógica de programación, la gestión del estado y la correcta estructuración del código.

A lo largo de este documento se detallará el proceso de desarrollo de los ejercicios propuestos, se evaluará el cumplimiento de criterios como la funcionalidad de la aplicación, la calidad del código, la organización del contenido y la claridad en la presentación de los resultados. De esta manera, se busca no solo demostrar la correcta implementación de una aplicación móvil, sino también fortalecer las competencias en análisis, redacción técnica y buenas prácticas de desarrollo.

II. Desarrollo

II.1 Ejercicios

1. En un supermercado un cajero captura los precios de los artículos que los clientes compran e indica a cada cliente cual es el monto de lo que deben pagar. Al final del día le indica a su supervisor cuanto fue lo que cobro en total a todos los clientes que pasaron por su caja.

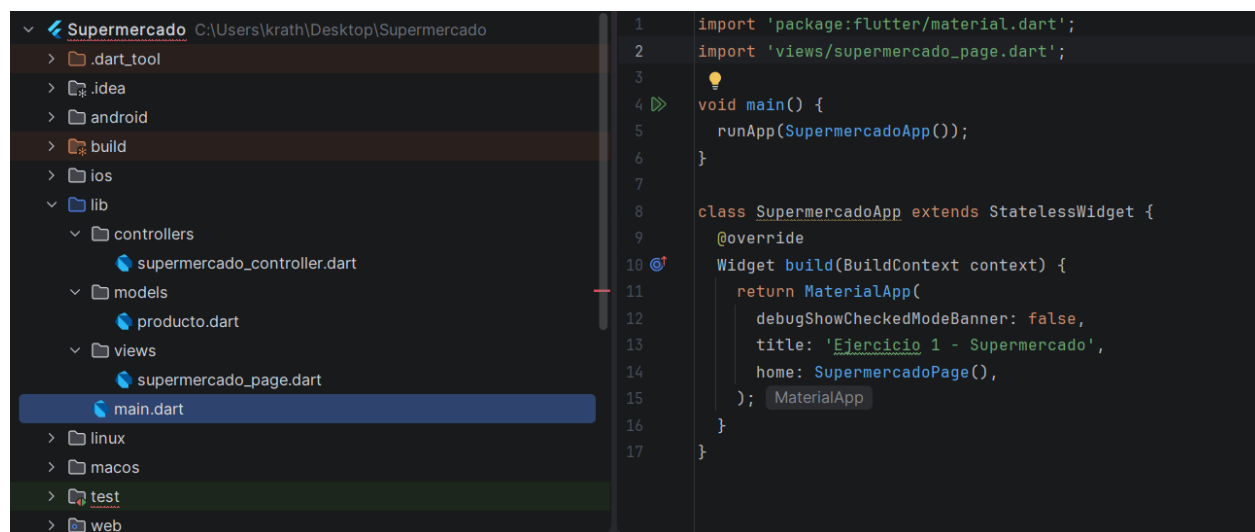


Figura 1: Estructura y main.dart.

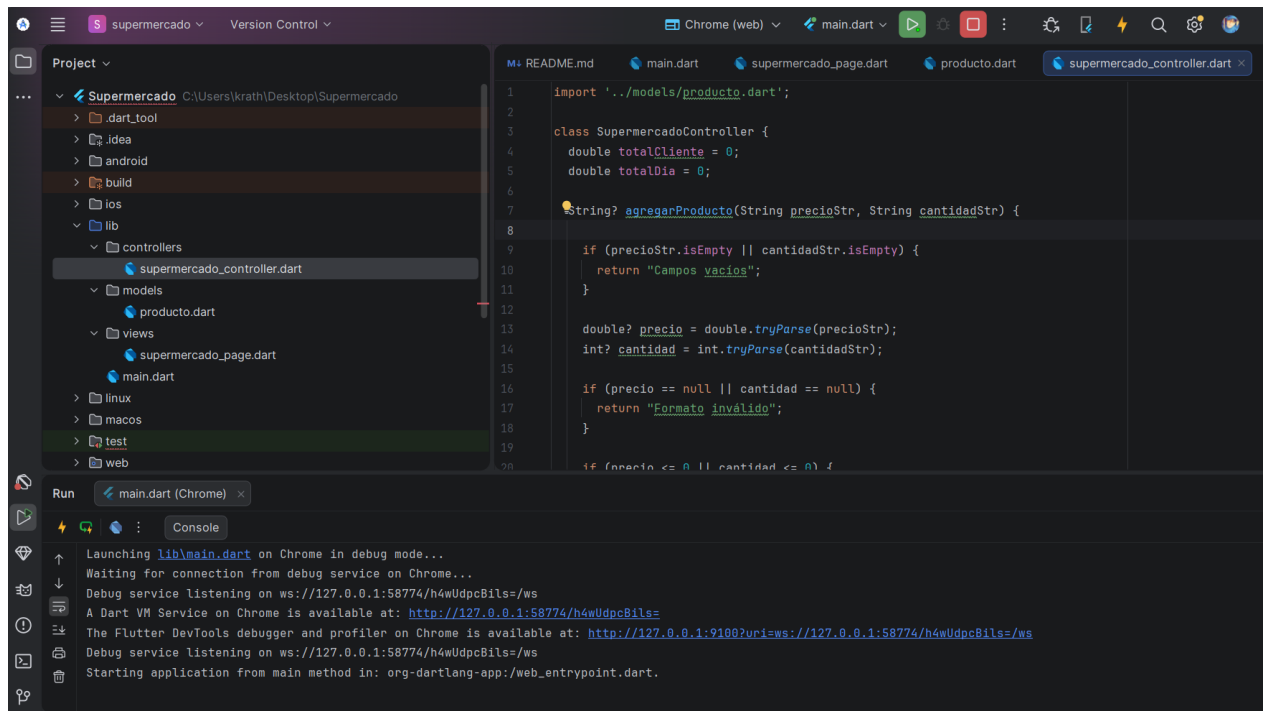


Figura 2: Ejecución y funcionamiento del aplicativo supermercado.

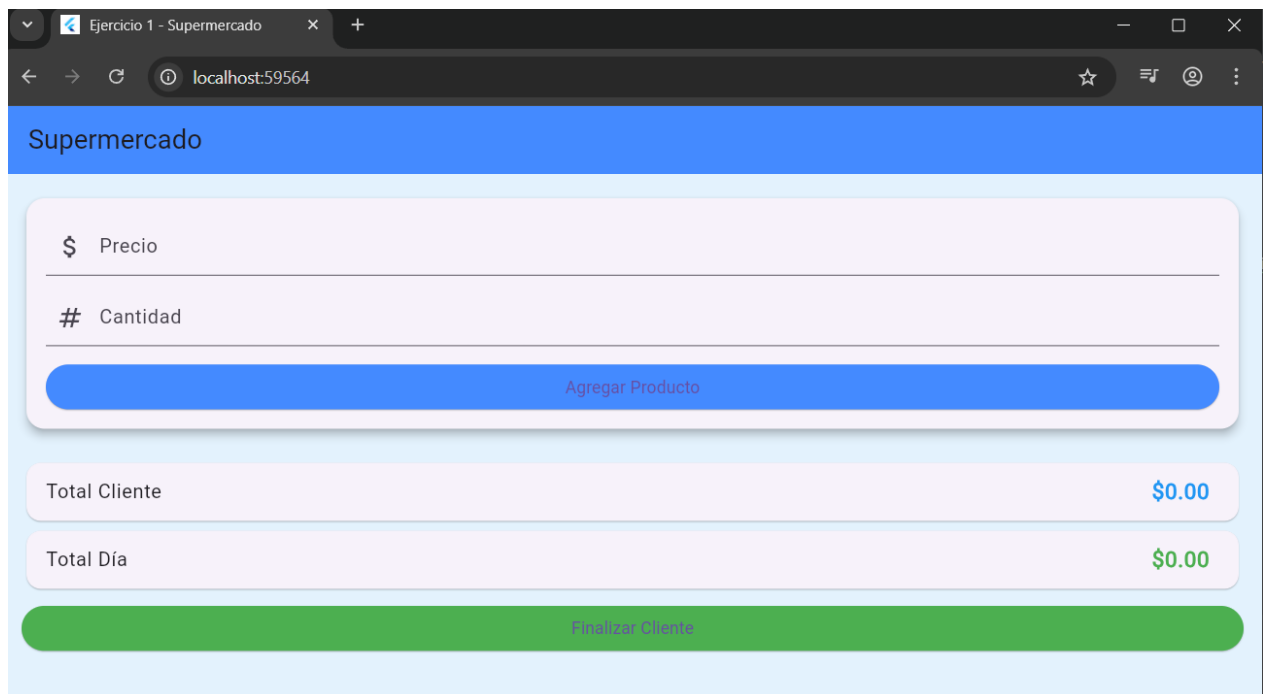


Figura 3: Aplicativo funcionando sin ningun problema.

The screenshot shows a web browser window with the title 'Ejercicio 1 - Supermercado' and the address 'localhost:59564'. The application has a blue header with the word 'Supermercado'. Below the header, there are two input fields. The first field is labeled 'Precio' with a '\$' symbol and contains the value '0'. Below it, the text 'Mayor a 0' is displayed in red. The second field is labeled 'Cantidad' with a '#' symbol and contains the value '0'. Below it, the text 'Mayor a 0' is also displayed in red. A blue button labeled 'Agregar Producto' is positioned below the second field. Below the input fields, there are two summary rows: 'Total Cliente' with a value of '\$0.00' and 'Total Día' with a value of '\$0.00'. A green button labeled 'Finalizar Cliente' is located below these rows. At the bottom of the page, a dark grey bar contains the text 'No hay productos'.

Figura 4: *validaciones de los campos.*

This screenshot shows the same web application as Figure 4, but with valid data entered. The 'Precio' field now contains '200' and the 'Cantidad' field contains '1'. The 'Total Cliente' remains '\$0.00', but the 'Total Día' has updated to '\$200.00'. The 'Agregar Producto' and 'Finalizar Cliente' buttons remain visible. The 'No hay productos' message is still present at the bottom.

Figura 5: *llenar los campos con datos validos y funcionamiento correcto.*

Este problema simula el funcionamiento de una caja en un supermercado. El cajero va registrando los precios de los productos que compra cada cliente y calcula el total que ese cliente debe pagar. Para ello, se acumulan los valores de los artículos ingresados. Una vez que el cliente termina su compra, se muestra su total y ese monto se suma a un acumulador general que representa lo cobrado durante todo el día. Al final de la jornada, el sistema permite conocer cuánto dinero se recaudó en total por todos los clientes atendidos.

Link del repositorio = <https://github.com/Asmalkevin1999/Moviles30737.git>

2. Disponemos de una máquina que puede dar vuelto con 5 tipos de monedas distintas: \$2, \$1, \$0,50, \$0,25 y \$0,10. Realizar el programa que dados el precio del artículo y la cantidad entregada por el consumidor nos indique el vuelto a entregar empleando el menor número posible de monedas.

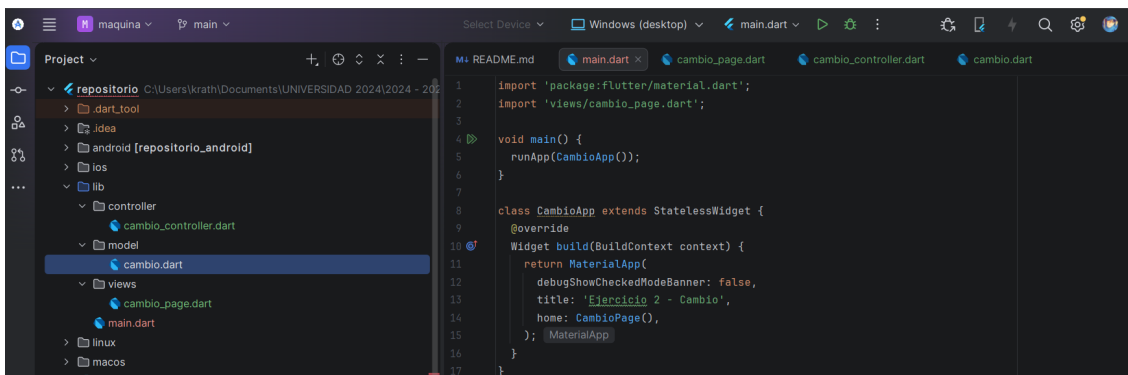


Figura 6: Estructura y main.dart.

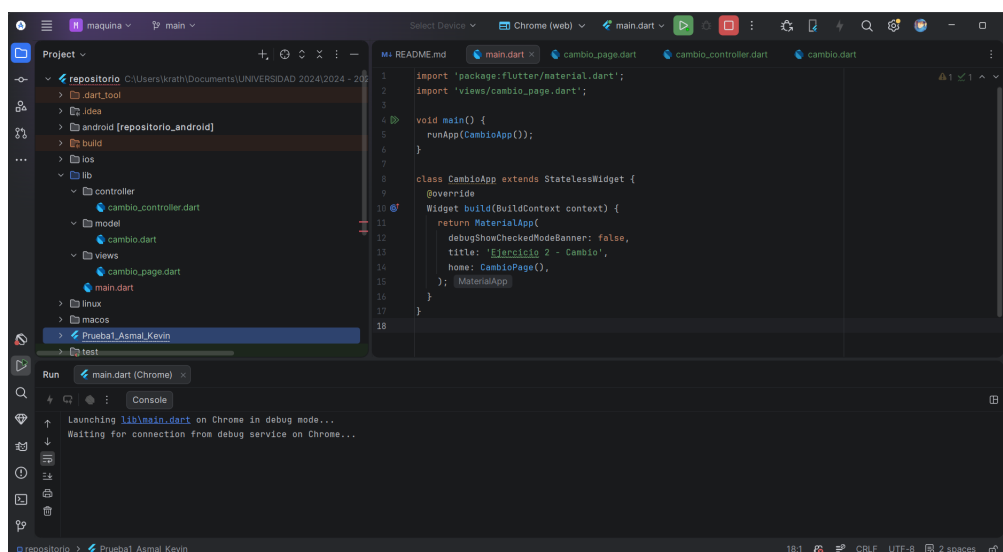


Figura 7: Ejecución y funcionamiento del aplicativo Máquina que puede dar vueltos.

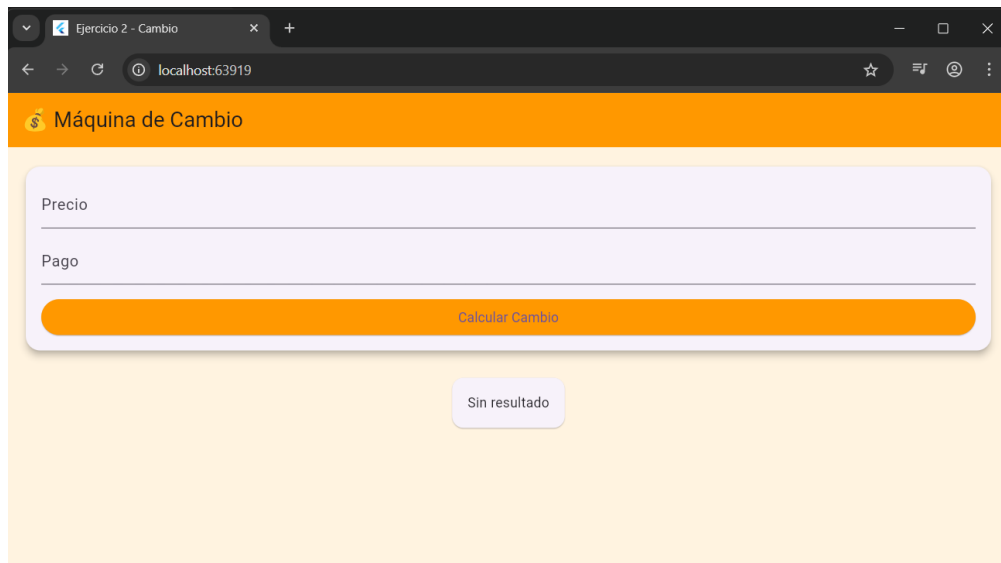


Figura 8: *Aplicativo funcionando sin ningun problema.*

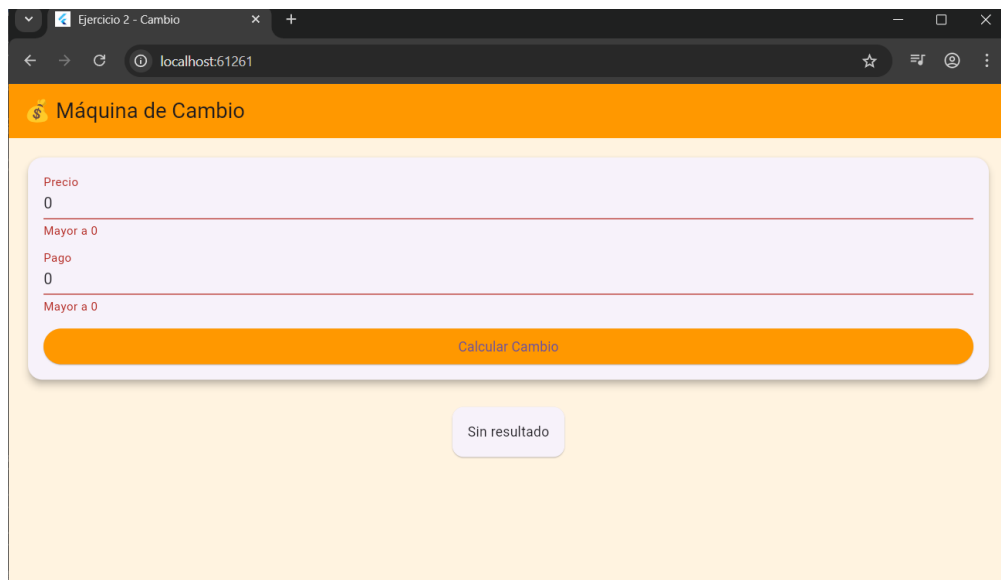


Figura 9: *Validaciones de los campos.*

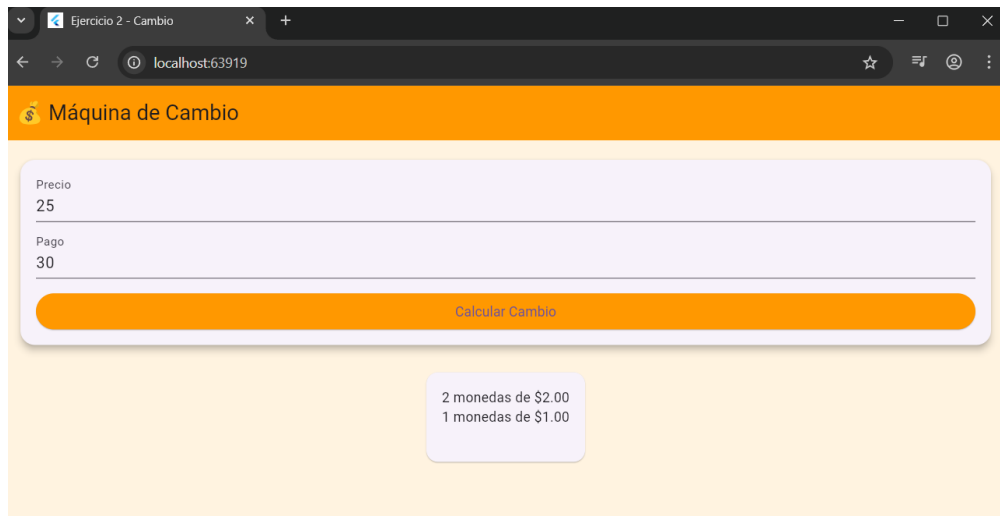


Figura 10: *llenar los campos con datos validos y funcionamiento correcto.*

Este ejercicio consiste en calcular el cambio que debe devolverse a un cliente después de realizar un pago, utilizando la menor cantidad posible de monedas disponibles. A partir del precio del artículo y el dinero entregado por el cliente, se obtiene el valor del vuelto y luego se descompone ese monto en monedas de diferentes denominaciones (\$2, \$1, \$0,50, \$0,25 y \$0,10). El objetivo es usar un método eficiente (generalmente un enfoque voraz) que seleccione primero las monedas de mayor valor para minimizar la cantidad total de monedas entregadas.

Link del repositorio = <https://github.com/Asmalkevin1999/Moviles30737.git>

3. Un año es bisiesto si es múltiplo de 4, exceptuando los múltiplos de 100, que sólo son bisiestos cuando son múltiplos además de 400, por ejemplo el año 1900 no fue bisiesto, pero el año 2000 si lo fué. Hacer un programa que dado un año A nos diga si es o no bisiesto.

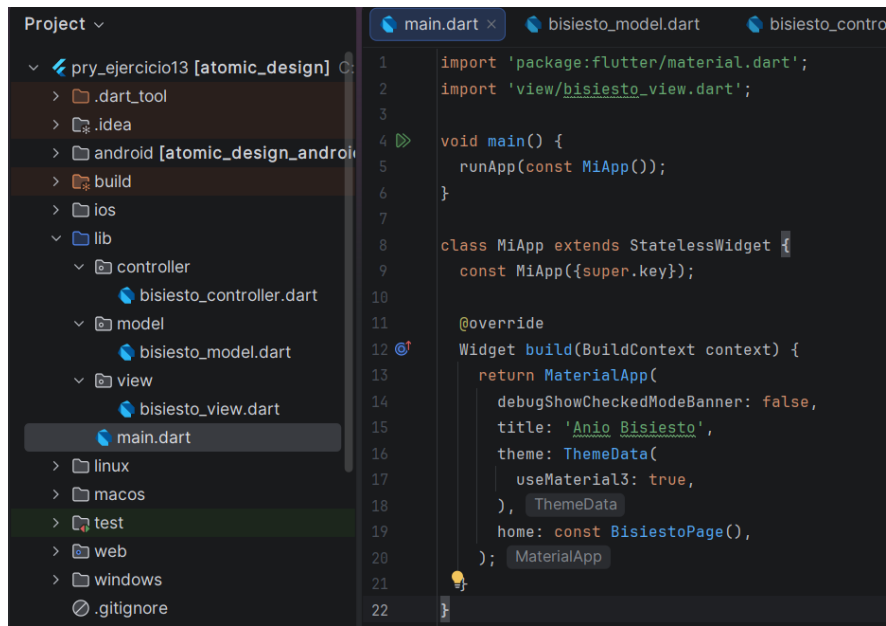


Figura 11: Estructura y main.dart.

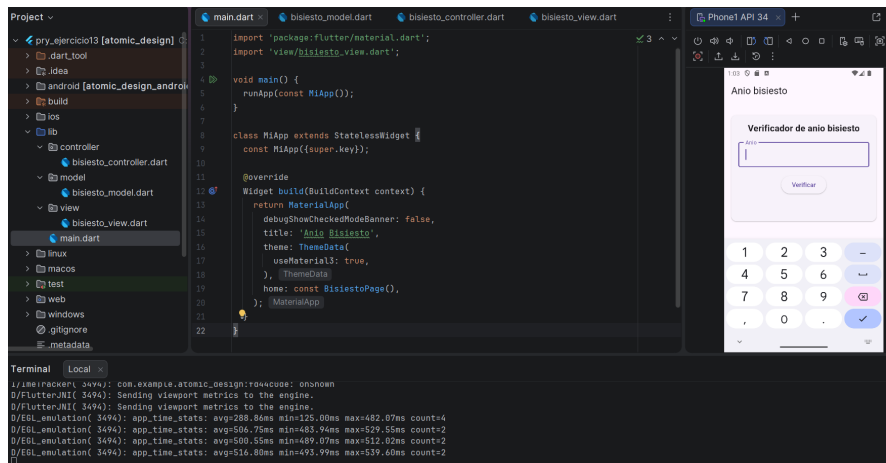


Figura 12: Ejecución del proyecto.

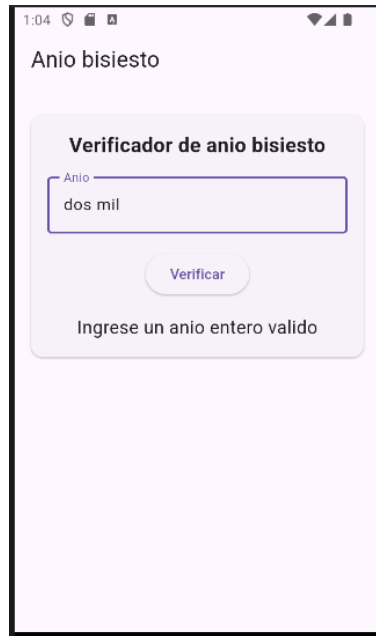


Figura 13: *Validación de entradas.*



Figura 14: *Comprobación de año no bisiesto.*

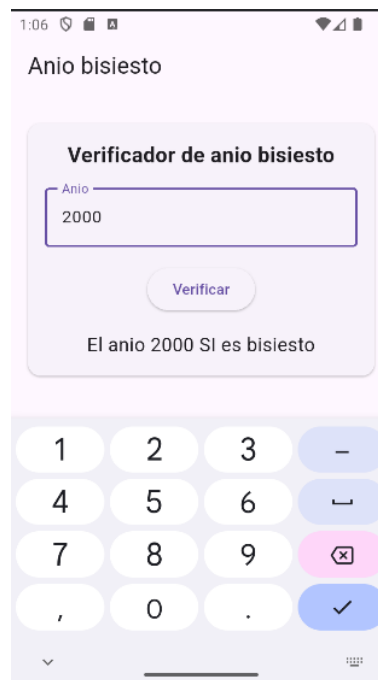


Figura 15: *Comprobación de año bisiesto.*

El problema consiste en determinar si un año dado es bisiesto aplicando una serie de condiciones matemáticas específicas. Como dato de entrada se tiene un número entero A que representa el año, y como salida se espera un mensaje que indique si dicho año es bisiesto o no. La lógica del problema establece que un año es bisiesto si es divisible entre 4; sin embargo, existe una excepción: si el año también es divisible entre 100, entonces deja de ser bisiesto, a menos que además sea divisible entre 400, en cuyo caso sí se considera bisiesto. Por lo tanto, la solución requiere evaluar condiciones combinadas de divisibilidad utilizando operadores lógicos.

Link del repositorio = <https://github.com/Pa004/Anio-Bisiesto.git>

4. Dados dos números enteros positivos N y D , se dice que D es un divisor de N si el resto de dividir N entre D es 0. Se dice que un número N es perfecto si la suma de sus divisores (excluido el propio N) es N . Por ejemplo 28 es perfecto, pues sus divisores (excluido el 28) son: 1, 2, 4, 7 y 14 y su suma es $1+2+4+7+14=28$. Hacer un algoritmo que dado un número N nos diga si es o no perfecto.

```

18 final bool verificacion;
19 final divicibles;
20
21 EjercicioModel({
22   required this.suma,
23   required this.verificacion,
24   required this.divicibles
25 });
26
27 static EjercicioModel calculos(int a){
28   final divicibles = <int>[];
29   for(int i=1; i<=a; i++){
30     if(a%i==0){
31       divicibles.add(i);
32     }
33   }
34
35   final suma = divicibles.reduce((x,y)=>x+y);
36
37   final bool verificacion = (suma==a);
38
39   return EjercicioModel(suma: suma, verificacion: verificacion, divicibles: divicibles);
40 }

```

Figura 16: Código modelo ejercicio 4.

```

1 class EjercicioController {
2   String procesar(String sa, String sb){
3     try{
4
5       final a=int.parse(sa);
6
7       final r = EjercicioModel.calculos(a);
8
9       return '''
10      Resultados:
11      Numero 1: ${a}
12      Divicibles del numero 1: ${r.divicibles}
13      Suma Divicibles: ${r.suma}
14      Verificacion: ${r.verificacion ? "El número 1 es perfecto":"El número 1 no es perfect"}
15
16      ''';
17
18     } catch(e){
19       return "Ingresar valores validos";
20     }
21   }
22 }

```

Figura 17: Código controlador ejercicio 4.

```

100
101 class NumerosPage extends StatelessWidget{
102   NumerosPage({super.key});
103
104   @override
105   Widget build(BuildContext context) {
106     return Scaffold(
107       appBar: AppBar(
108         title: Text('Números Perfectos'),
109         backgroundColor: Colors.greenAccent,
110       ),
111       body: Padding(
112         padding: EdgeInsets.all(20),
113         child: AnalisisCard(),
114       ),
115     );
116   }
117 }
118
119
120
121 }

```

Figura 18: Código vista ejercicio 4.

Números Perfectos

Ingrese 1 número entero

A
28

Calcular

Resultados:
Numero 1: 28
Divisibles del numero 1: [1, 2, 4, 7, 14]
Suma Divisibles: 28
Verificacion: El número 1 es perfecto

Figura 19: Ejecucion ejercicio 4

5. Cinco miembros de un club contra la obesidad desean saber cuanto han bajado o subido de peso desde la ultima vez que se reunieron. Para esto se debe realizar un ritual de pesaje en donde cada uno se pesa en diez básculas distintas para así tener el promedio mas exacto de su peso. Si existe diferencia positiva entre este promedio de peso y el peso de la ultima vez que se reunieron, significa que subieron de peso. Pero si la diferencia es negativa, significa que bajaron. Lo que el problema requiere es que por cada persona se imprima un letrero que diga: "SUBIO" o "BAJO" y la cantidad de kilos que subió o bajo de peso

```

1 class Member {
2     final String name;
3     final double lastWeight;
4     final List<double> currentWeights;
5
6     Member({
7         required this.name,
8         required this.lastWeight,
9         required this.currentWeights,
10    });
11
12    // Lógica de cálculo dentro del modelo
13    double get averageWeight =>
14        currentWeights.reduce((a, b) => a + b) / currentWeights.length;
15
16    double get weightDifference => averageWeight - lastWeight;
17
18    String get status => weightDifference > 0 ? "SUBIÓ" : "BAJÓ";
19 }

```

Figura 20: Código modelo ejercicio 5.

```

1 import 'package:tarea-creacion-movil/model/member_model.dart';
2
3 class MemberController {
4     // Datos de ejemplo para los 5 miembros
5     List<Member> getMembers() {
6         return [
7             Member(
8                 name: "Miembro 1",
9                 lastWeight: 80.0,
10                currentWeights: [81, 82, 80, 81, 82, 81, 80, 81, 82, 81],
11            ),
12            Member(
13                name: "Miembro 2",
14                lastWeight: 95.0,
15                currentWeights: [94, 93, 94, 94, 92, 93, 94, 93, 94, 93],
16            ),
17            Member(
18                name: "Miembro 3",
19                lastWeight: 70.0,
20                currentWeights: [72, 71, 72, 73, 72, 72, 71, 72, 73, 72],
21            ),
22            Member(
23                name: "Miembro 4",
24                lastWeight: 110.0,
25                currentWeights: [108, 109, 107, 108, 108, 109, 108, 107, 108, 108],
26            ),
27        ];
28    }
29 }

```

Figura 21: Código controlador ejercicio 5.

```

1 itemBuilder: (context, index) {
2     final member = members[index];
3     final diff = member.weightDifference.abs();
4
5     return Card(
6         margin: EdgeInsets.all(10),
7         child: ListTile(
8             title: Text(
9                 member.name,
10                style: TextStyle(fontWeight: FontWeight.bold),
11            ),
12            subtitle: Text(
13                "Promedio actual: ${member.averageWeight.toStringAsFixed(2)} kg",
14            ),
15            trailing: Column(
16                mainAxisAlignment: MainAxisAlignment.center,
17                children: [
18                    Text(
19                        member.status,
20                        style: TextStyle(
21                            color: member.status == "SUBIÓ"
22                                ? Colors.red
23                                : Colors.green,
24                            fontWeight: FontWeight.bold,
25                        ),
26                    ),
27                ],
28            ),
29        ),
30    );
31 }

```

Figura 22: Código vista ejercicio 5.

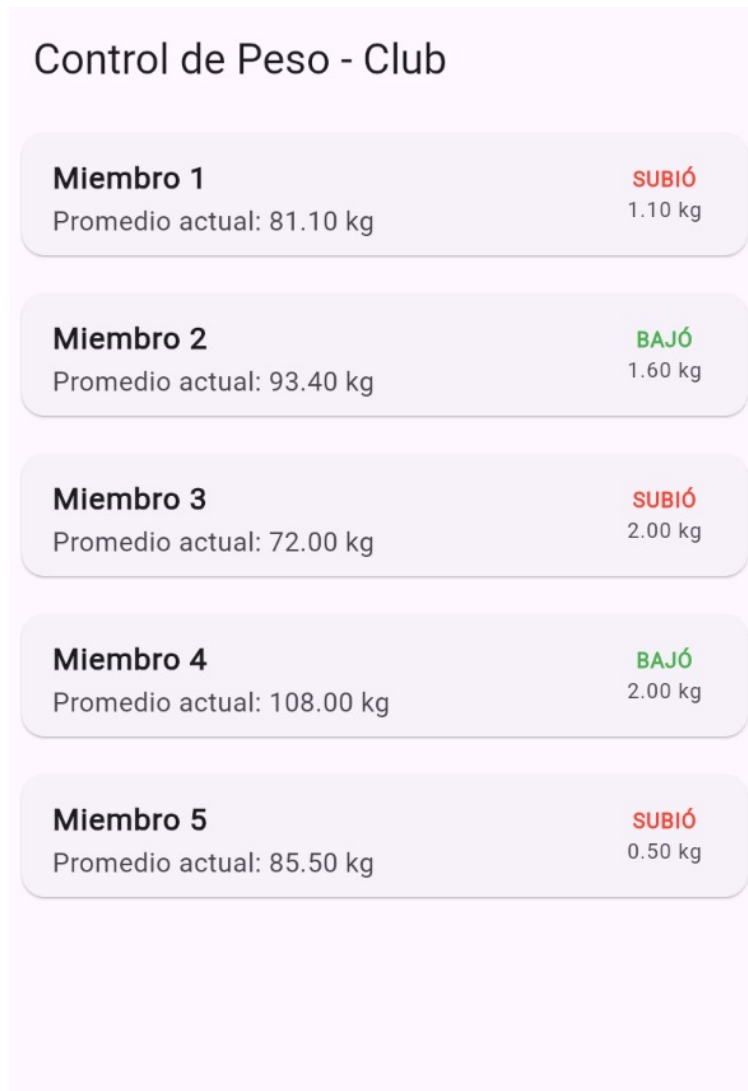


Figura 23: Ejecucion ejercicio 5

III. Conclusiones

- El desarrollo de la aplicación permitió aplicar de manera práctica los conceptos fundamentales de programación móvil, especialmente en la implementación de lógica condicional y control de flujo para resolver problemas como la determinación de años bisiestos.
- Se evidenció la importancia de un buen diseño de interfaz (UI/UX), ya que una estructura clara y organizada facilita la interacción del usuario y mejora la comprensión del funcionamiento de la aplicación.
- La correcta organización del código y el uso de buenas prácticas de programación contribuyeron a obtener una aplicación funcional, mantenible y coherente con los requerimientos

planteados en la actividad.

- El proceso de análisis, desarrollo y documentación fortaleció habilidades técnicas y de redacción, permitiendo presentar resultados de forma clara, estructurada y acorde a los criterios de evaluación establecidos.

IV. Referencias

- Google. (2024). Flutter documentation. <https://docs.flutter.dev/>
- Oracle. (2023). The Java™ Tutorials: Language Basics. <https://docs.oracle.com/javase/tutorial/>
- Microsoft. (2024). Introducción a C#. <https://learn.microsoft.com/es-es/dotnet/csharp/>
- W3Schools. (2024). Programming Tutorials and References. <https://www.w3schools.com/>
- Pressman, R. S., & Maxim, B. R. (2020). Ingeniería de software: Un enfoque práctico (9.^a ed.). McGraw-Hill.
- Sommerville, I. (2021). Ingeniería de software (10.^a ed.). Pearson.